Vibe Coding: Building Real Apps *with AI*

Five Thursday evenings, no prerequisites, twelve people. We build **HaverTrack** — a calorie and nutrition tracker for Haverford students — and put it on real phones before Fall Break.

Thursdays, 6:00–7:30 PM · Sept 10 to Oct 8 · Location TBD.

Akramjon Abdurakhimov · aabdurakhi@haverford.edu · class channel for everything else.

Half of each night is new material, half is building together. Between sessions everyone builds the same feature, alone — roughly three or four hours of work a week.

Use whatever AI tool you like on your own projects. For HaverTrack we drive with Claude Code.

Calendar

Thu Sep 10	**01**	What vibe coding is, the tools, campus problems, HaverTrack, GitHub
Thu Sep 17	**02**	Directing the agent, running HaverTrack, and making a screen look decent
Thu Sep 24	**03**	Where the data lives — databases, Supabase, who can see what
Thu Oct 1	**04**	APIs, secrets, and the outside world
Thu Oct 8	**05**	Test it on people, then ship it — the last Thursday before Fall Break

How the weekly build works

Every week the whole class builds the *same* feature, on your own branch, from the same short spec. We open the next session by putting all twelve versions side by side on the projector — then merge the one that's genuinely good, or the best pieces of several.

- **Same spec for everyone**, handed out at the end of each session
- **Build it alone.** Ask for help in the channel — just not for somebody's code
- **Pull request open by Wednesday night** so they're queued up for Thursday
- **After we merge, pull** `main` before you start the next week

Your version not getting merged is the normal outcome, not a failure — twelve attempts at one problem in one room is the fastest way to see what a good prompt, a good decision, and a good screen actually look like.

---

01 What this is, what to use, and what we're building

Thu Sep 10

- **What vibe coding actually is.** What a language model does under the hood — it predicts, it doesn't know, which is why it is confidently wrong. Where the term came from: Andrej Karpathy, February 2025, who meant it for throwaway weekend projects. We're going to do the opposite and keep one alive past the end of the class.
- **What people have built this way** — a few real examples, including the ones that fell over, and what separates the two.
- **The tools.** Agents in a terminal: **Claude Code**, **Codex**. Agent-native editors: **Cursor**, **Antigravity**, **Zcode**. Where you type: **Terminal**, **Ghostty**. Where code lives: **GitHub**. Design: **Figma** to make it, **Mobbin** to steal from. Live: the same prompt in two tools, side by side.
- **Problems on campus.** Open floor — not teams, not pitches. Say what's broken or annoying at Haverford. Everything said gets written down and becomes our running idea list.
- **HaverTrack.** A calorie and nutrition tracker: log what you eat, build your own plate from today's DC menu or from anything else, set a goal, and see the macros and micronutrients behind it rather than one meaningless number. Live demo on a phone, including the parts that are still bad.
- **GitHub accounts.** Sign up, turn on two-factor (bring your phone), and what a repository actually is.
- **The class repo.** Everyone gets invited and clones it. What a pull request is, and why nothing goes into the app without one.

WEEK 1 · BEFORE SEP 17

- **Build a to-do app** with any vibe coding tool you like. It needs a design you'd be willing to show someone and at least one animation. Bring three things: the repo link, a 60-second screen recording, and one paragraph on the first thing your tool got wrong.
- **Get HaverTrack running on your own phone** — clone, `npm install`, `npx expo start`, scan the QR code with Expo Go.
- **Open a pull request** on the class repo with your name added to `docs/students.md`. It's a throwaway change — the point is that you've done the whole loop once.

02 Directing the agent

Thu Sep 17

- To-do app show and tell — two minutes each. What each tool was good at, and where it fell apart
- Writing a prompt an agent can actually build from: be specific, name the files, say what is *out* of scope
- Plan first, code second. Reviewing the plan instead of arguing with the result
- Giving the agent memory: `CLAUDE.md`, `AGENTS.md`, skills and MCP — how our repo tells it what it needs to know before it writes a line
- A tour of HaverTrack: the folders, the screens, and where things live
- Making a screen look decent: Figma and Mobbin for references, then hierarchy, spacing, touch targets and contrast — the few rules that fix most screens
- Design tokens: colors and type live in one file, and no screen hardcodes anything
- Everyone opens their first pull request before leaving

WEEK 2 · EVERYONE BUILDS: THE DAILY SUMMARY CARD

The thing you see first when you open the app: calories and macros eaten today against your targets, at the top of the home screen.

- Reads real logged meals, and still looks right when nothing has been logged yet
- One tap from the card to logging something
- Colors and type come from the theme file — nothing hardcoded
- Holds up at the extremes: nothing eaten yet, and wildly over on one macro

Thursday we compare all twelve on clarity at a glance, whether it looks like it belongs in the app, and what it does at the edges.

03 Where the data lives

Thu Sep 24

- **The bake-off.** Twelve daily summary cards on the projector, five minutes of comparing, and we merge the best one — or the best parts of three
- What a database is, in plain terms — tables, rows, ids — using ours: `profiles`, `meal_logs`, `meal_log_items`, `menu_items`, `weight_entries`
- The handful of SQL you actually need. There isn't much
- Supabase: everyone makes a project and points the app at it
- Migrations — why we change the database with files instead of clicking around in a dashboard
- **Row Level Security.** The database itself refuses to hand out other people's rows. We'll switch it off and read each other's food logs, then switch it back on
- Signing up, the six-digit email code, staying signed in

WEEK 3 · EVERYONE BUILDS: RATE AND NOTE A MEAL

Give anything you logged a rating out of five and a short note, then find your best dishes at the DC again later.

- A new table and a policy, so nobody can read your notes but you
- A migration file that runs cleanly on an empty database
- Rating and note attach to a meal you already logged
- A screen listing your highest-rated dishes

Thursday we compare the schemas, then try to read each other's notes from a second account. Anything that leaks doesn't get merged.

04 APIs, secrets, and the outside world

Thu Oct 1

- **The bake-off.** Twelve versions of rate-and-note. We compare the schemas first, then the screens, then try to break the policies
- What an API is: request, response, JSON, and the status codes worth recognizing
- Live — we call the Nutrislice endpoint from the terminal and read the raw menu data that becomes tonight's dinner list
- How the DC menu gets into the app: a scheduled GitHub Action that runs three times a day, before each meal
- Why an API key can never live inside a phone app, and what an Edge Function is for
- Camera and barcode scanning; turning a photo of a plate into something you can edit and save
- What your screen shows when it's empty, loading, offline, or simply broken — all four, every screen

> WEEK 4 · EVERYONE BUILDS: "WHAT SHOULD I EAT?"
>
> Today's real DC menu, narrowed to what actually suits you right now.
>
> - Filters by the dietary tags that come from the live feed — vegan, vegetarian, allergens
> - And by what still fits in your calories and macros for the day
> - All four states answered: empty, loading, offline, error
> - No keys or secrets anywhere in the app
>
> Thursday we run all twelve against today's actual menu, then against a morning where the sync failed. The one that survives both gets merged and ships.

05 Test it on people, then ship it

Thu Oct 8

- **The last bake-off.** Twelve takes on "what should I eat?", judged against today's real menu. Whatever we merge tonight is what ships tonight
- **Usability testing, live.** Hand your phone to someone, give them one task, and say nothing. You are not allowed to explain your own app — every explanation is a problem you're hiding
- Fix the worst thing each team hears, in the room, in twenty minutes
- Pre-flight: types, a clean build, migrations applied, environment variables in place
- Expo Go versus a real build; TestFlight and the web version; what a release actually is
- We cut the release together, and students from outside the class install it in the room
- Demos — five minutes each: one thing you shipped, one thing you cut, one thing that broke
- Who keeps it alive after the class ends, and how the next group picks it up

> AFTER
>
> Fall Break starts the next afternoon and people take the app home with them. Watch the channel — the first real bug reports arrive that weekend, and answering one is the last thing this class asks of you.

Bring to Session 1

- A laptop you can install things on. macOS or Windows — a Chromebook won't work
- Your phone, with an authenticator app, for GitHub's two-factor step
- A GitHub account already made, if you can manage it beforehand
- Node and Git installed, if you get that far. We'll sort out anyone who doesn't
- One AI coding tool installed and signed in — any of them

Four ground rules

- **Use AI for everything.** That is the class, not a loophole in it
- **Don't ship what you can't explain** — not for me, for whoever picks it up next, which will probably be you
- **Ask before you lose an hour.** Stuck on the same error for sixty minutes is not persistence. Post it in the channel
- **This app holds other students' food logs, weight and goals.** Treat that data like it's yours, because before long it will be

In five weeks you'll have written a database migration, worked against a live API, and cut a release build — things most computer science majors don't touch until their third year. The syntax was never the hard part.